



Target: CruShead1999 – Eugenio D. Deluca (KarasuRØØT) – Date: 14/08/2024

Hey there! Well, the purpose of this tutorial is to try and solve this old crackme that taught many of us back when we were starting out.

Specifically, Ricardo Narvaja showed how to solve it using Olly in his old tutorials—in INTRODUCTION TO CRACKING WITH OLLYDBG PART 16. Although he shows how to reach a solution, my goal is to approach it again after all this time and create a keygen so it works with any name.

I'm going to stick with Olly—obviously, the path is identical to Ricardo's original tutorial up to a certain point.

Alright, first I'm going to set a BP on `GetDlgItemTextA`—this is because, according to MSDN, it 'retrieves the title or text associated with a control in a dialog box.

OllyDbg - CRACKME.EXE

File View Debug Plugins Options Window Help

LEMTWHC / KBR... S

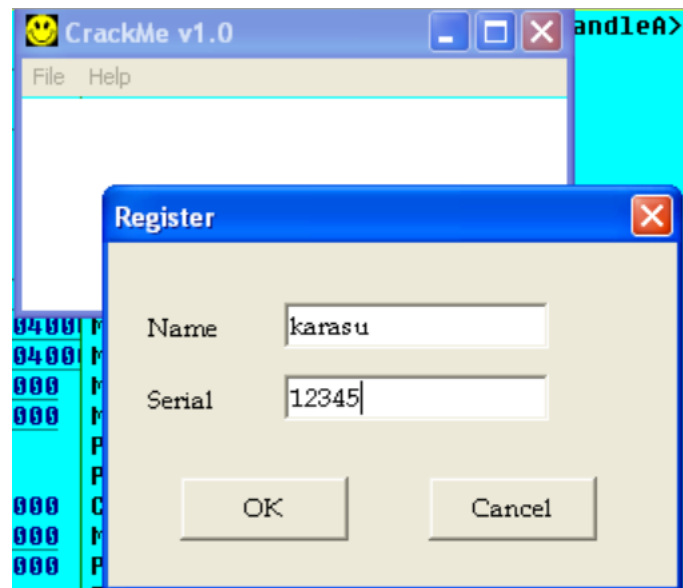
CPU - main thread, module CRACKME

Address	Hex dump	ASCII
00402000	00 00 00 00 00 00 00 00	
00402008	00 00 00 00 00 00 00 00	
00402010	00 00 00 00 00 00 00 00	
00402018	00 00 00 00 00 00 00 00	
00402020	00 00 00 00 00 00 00 00	
00402028	00 00 00 00 00 00 00 00	
00402030	00 00 00 00 00 00 00 00	

Command: BP GetDlgItemTextA BP address, string – Break with condition

Program entry point

We hit RUN and get to the corresponding window -> if we click on the 'Help' -> 'Register' option, the famous registration screen appears, asking us for a name and a serial.

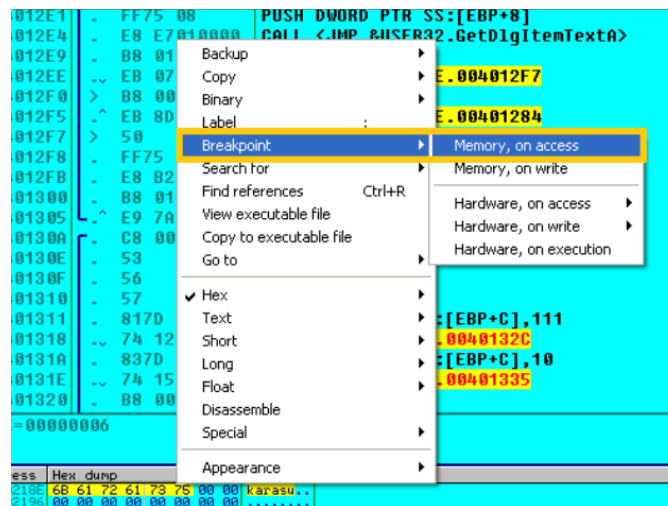


So I'll start testing—I'll use 'karasu' as the name and '12345' as the serial, and then I'll hit OK. It stops at the `MOV EDI,EDI` instruction -> but the important thing here is what appears on the STACK -> that's where the input for the Name is stored—meaning 'karasu'. Alright, let's move on -> Right-click -> Follow in Dump. If I look at the dump window, it's at 0—this is because, as Ricardo explains, it hasn't written anything yet; the API hasn't executed yet. We hit Ctrl + F9, or go to the top menu -> Debug -> Execute till return.

7E3EB084	74 06	JE SHORT USER32.7E3EB08C
7E3EB086	8B45 10	MOV EAX,DWORD PTR SS:[EBP+10]
7E3EB089	C600 00	MOV BYTE PTR DS:[EAX],0
7E3EB08C	33C0	XOR EAX,EAX
7E3EB08E	5D	POP EBP
7E3EB08F	C2 1000	RETN 10
7E3EB092	90	NOP
7E3EB093	90	NOP
7E3EB094	90	NOP
7E3EB095	90	NOP
7E3EB096	90	NOP
7E3EB097	8BFF	MOV EDI,EDI
Return to 004012C9 (CRACKME.004012C9)		
Address	Hex dump	ASCII
0040218E	68 61 72 61 73 75 00 00	karasu..
00402196	00 00 00 00 00 00 00 00	
0040219E	00 00 00 00 00 00 00 00	
004021A6	00 00 00 00 00 00 00 00	
004021AE	00 00 00 00 00 00 00 00	
004021B6	00 00 00 00 00 00 00 00	
004021BE	00 00 00 00 00 00 00 00	
004021C6	00 00 00 00 00 00 00 00	
Command BP GetDlgItemTextA		

From this point on, I'm going to deviate a bit from what was shown in Ricardo's original tutorial.

Now we can see that it wrote the name—OK -> F7—now what I'm going to do is highlight the hexadecimal value in the dump—I'll mark 6B 61 72 61 73 75—Then I'm going to set a Breakpoint on Access -> to do this -> right-click ->



And we hit F9 or RUN (x2, meaning twice, to reach the instruction) ->

Let's remember that it stops one instruction before—meaning that `MOV AL, BYTE PTR DS:[EDI]` hasn't executed yet -> if we look at Olly's comments, it indicates that `[004018E]` holds 6B (which is the hex value for 'k' in the ASCII table) and it gets saved into AL—I keep tracing and reach this point, which I believe is important:

OllyDbg - CRACKME.EXE - [CPU - main thread, module CRACKME]

File View Debug Plugins Options Window Help

004013C1 > C3 RETN

004013C2 \$ 33FF XOR EDI,EDI

004013C4 - 33DB XOR EBX,EBX

004013C6 > 8A1E MOV BL,BYTE PTR DS:[ESI]

004013C8 - 84DB TEST BL,BL

004013CA ~ 74 05 JE SHORT CRACKME.004013D1

004013CC - 03FB ADD EDI,EBX

004013CE - 46 INC ESI

004013CF ^ EB F5 JMP SHORT CRACKME.004013C6

004013D1 > C3 RETN

004013D2 \$ 2C 20 SUB AL,20

004013D4 - 8806 MOV BYTE PTR DS:[ESI],AL

004013D6 - C3 RETN

004013D7 - C3 RETN

004013D8 \$ 33C0 XOR EAX,EAX

004013DA - 33FF XOR EDI,EDI

004013DC - 33DB XOR EBX,EBX

004013DE - 8B7424 04 MOV ESI,DWORD PTR SS:[ESP+4]

004013E2 > B0 0A MOV AL,0A

004013E4 - 8A1E MOV BL,BYTE PTR DS:[ESI]

004013E6 - 84DB TEST BL,BL

004013E8 ~ 74 0B JE SHORT CRACKME.004013F5

004013EA - 80EB 30 SUB BL,30

004013ED - 0FAFF8 IMUL EDI,EAX

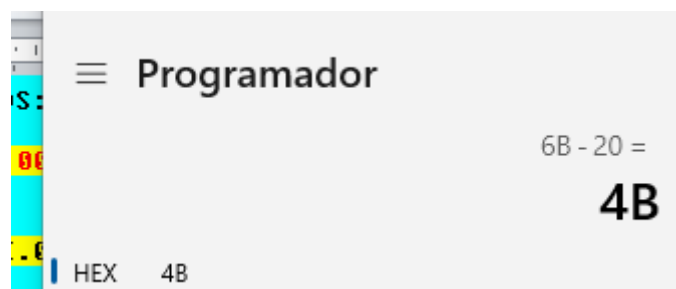
004013ED - 0FAFF8 ADD EDI,EBX

AL=4B ('K')

DS:[0040218E]=6B ('k')

Address	Hex dump	ASCII
0040218E	6B 61 72 61 73 75 00 00	karasu..

Let's remember that $AL = 6B \rightarrow$ then it subtracts 20 in `SUB AL, 20` so this means that AL is now equal to $4B$



And that means that 'k' is now $4B$ (which is 'K' in ASCII) and then there is a RETURN (RET) $\rightarrow$ so it goes back to $\rightarrow$ Return to 00401399 (CRACKME.00401399)

As we keep tracing -> I hit the loop again -> notice that now it takes the 'a' and AL retains the value 4B ('K')

Address	Disassembly	Comment
00401383	> 8A 06	MOV AL, BYTE PTR DS:[ESI]
00401385	. 84 C0	TEST AL, AL
00401387	~ 74 13	JE SHORT CRACKME.0040139C
00401389	. 3C 41	CMP AL, 41
0040138B	~ 72 1F	JB SHORT CRACKME.004013AC
0040138D	. 3C 5A	CMP AL, 5A
0040138F	~ 73 03	JNB SHORT CRACKME.00401394
00401391	. 46	INC ESI
00401392	^ EB EF	JMP SHORT CRACKME.00401383
00401394	> E8 39 00 00 00	CALL CRACKME.004013D2
00401399	. 46	INC ESI
0040139A	^ EB E7	JMP SHORT CRACKME.00401383
0040139C	> 5E	POP ESI
0040139D	. E8 20 00 00 00	CALL CRACKME.004013C2
004013A2	. 81 F7 78 56 00 00	XOR EDI, 5678
004013A8	. 8B C7	MOV EAX, EDI
004013AA	~ EB 15	JMP SHORT CRACKME.004013C1
004013AC	> 5E	POP ESI
004013AD	. 6A 30	PUSH 30
004013AF	. 68 60 21 40 00	PUSH CRACKME.00402160
004013B4	. 68 69 21 40 00	PUSH CRACKME.00402169
004013B9	. FF 75 08	PUSH DWORD PTR SS:[EBP+8]
004013BC	. E8 79 00 00 00	CALL <JMP.&USER32.MessageBoxA>
004013C1	> C3	RETN
004013C2	^ 33 FF	XOR EDI, EDI

DS:[0040218F]=61 ('a')

AL=4B ('K')

Jumps from 00401392, 0040139A

In the next instruction AL = 61, I keep tracing and once again:

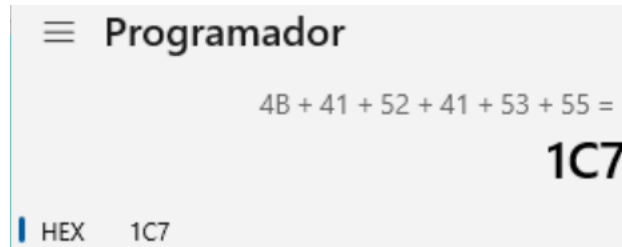
Address	Disassembly	Comment
004013D2	\$ 2C 20	SUB AL, 20
004013D4	. 8B 06	MOV BYTE PTR DS:[ESI], AL
004013D6	. C3	RETN
004013D7	. C3	RETN
004013D8	\$ 33 C0	XOR EAX, EAX
004013DA	. 33 FF	XOR EDI, EDI
004013DC	. 33 0B	XOR EBX, EBX
004013DE	. 8B 74 24 04	MOV ESI, DWORD PTR SS:[ESP+4]
004013E2	> B0 0A	MOV AL, 0A
004013E4	. 8A 1E	MOV BL, BYTE PTR DS:[ESI]
004013E6	. 84 DB	TEST BL, BL
004013E8	~ 74 0B	JE SHORT CRACKME.004013F5
004013EA	. 80 EB 30	SUB BL, 30
004013ED	. 0F AF F8	IMUL EDI, EAX
004013F0	. 03 FB	ADD EDI, EBX
004013F2	. 46	INC ESI
004013F3	^ EB ED	JMP SHORT CRACKME.004013E2
004013F5	> 81 F7 34 12 00 00	XOR EDI, 1234
004013FB	. 8B DF	MOV EBX, EDI
004013FD	. C3	RETN
004013FE	~ FF 25 84 31 40 00	JMP DWORD PTR DS:[<&USER32.KillTimer>]
00401404	~ FF 25 88 31 40 00	JMP DWORD PTR DS:[<&USER32.GetSystemMetrics>]
0040140A	\$~ FF 25 8C 31 40 00	JMP DWORD PTR DS:[<&USER32.LoadCursorA>]
00401410	~ FF 25 90 31 40 00	JMP DWORD PTR DS:[<&USER32.LoadAcceleratorA>]
00401416	\$~ FF 25 94 31 40 00	JMP DWORD PTR DS:[<&USER32.MessageBoxA>]

AL=41 ('A')

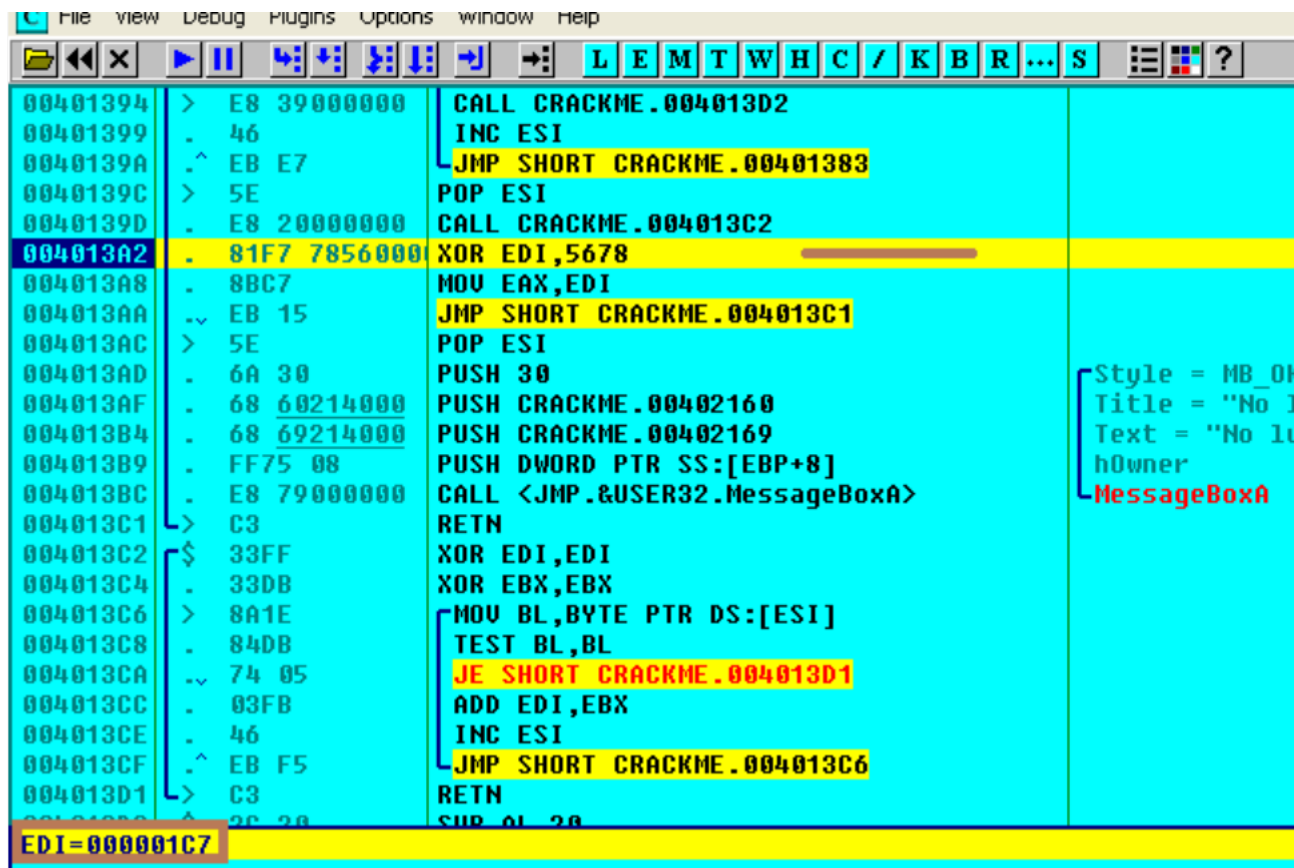
DS:[0040218F]=61 ('a')

In other words, it takes the next letter again and converts it to uppercase -> and then it keeps gathering the result in `EDI`—to keep it from getting tedious, what it does is loop through the string entered in the name -> puts it all in uppercase -> and then adds up each character—so if I entered 'karasu' -> it will basically convert it to 'KARASU' and then add up each character -> KARASU = 4B 41 52 41 53 55 (in Hex)

Then it adds it up character by character

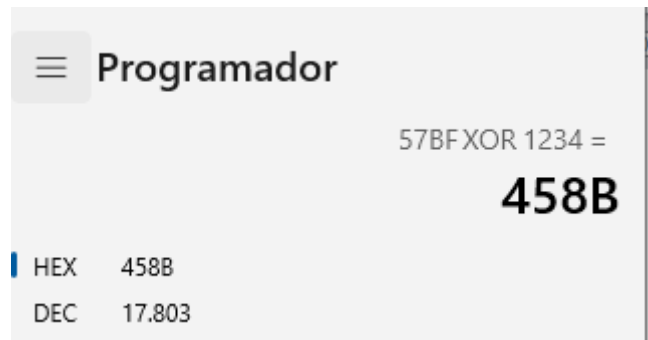


Once it loops through the entire string and exits the loop, we arrive here:

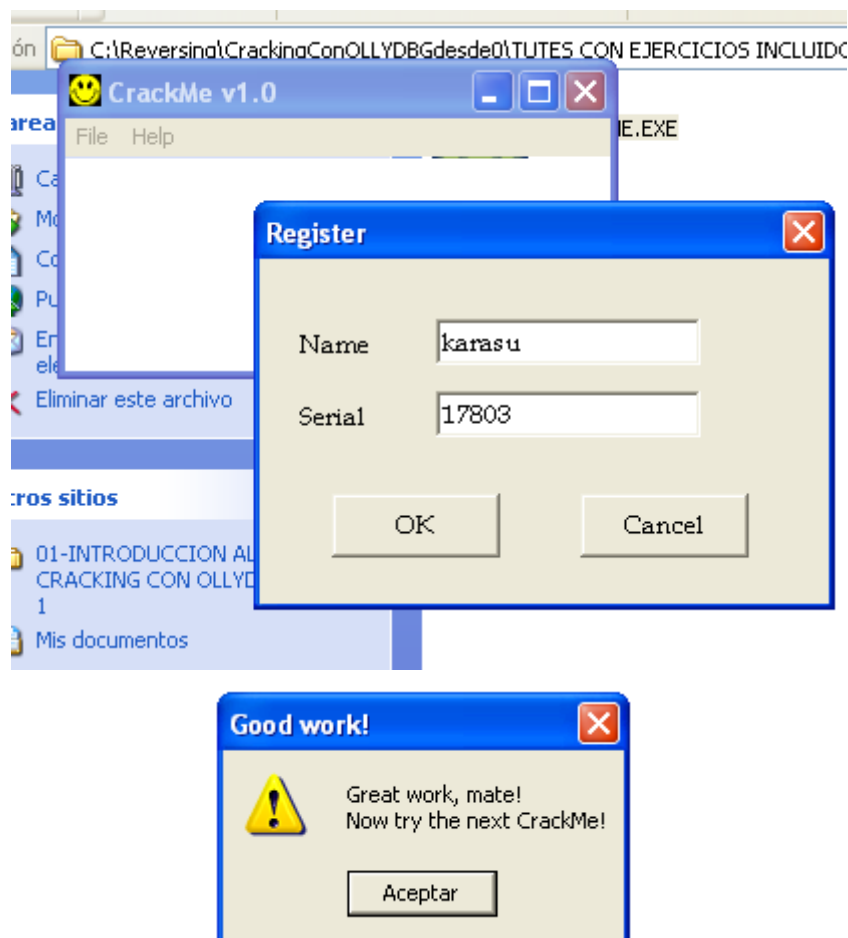


Notice that, sure enough, $EDI = 1C7 \rightarrow$ Then it does a $XOR\ EDI, 5678 = 57BF$ and subsequently moves that value into $EAX \rightarrow$

There, basically what it does is `XOR EDI, 1234`—so that means that IF `EDI = 57BF`
=> `57BF XOR 1234 =`



Meaning that now it will be equal to `458B` = which means that the serial would be `17803`—the program continues, and that's when it throws the bad boy message—but we can test it



Upon clicking OK -> it returns the message 'Great work, mate! Now try the next Crackme!' -> so we beat it :D—something that might be worth noting is that the key `17803` works for both the string `'KARASU'` and `'karasu'`.

Alright, it's time to write a script.

Reviewing then, basically what needs to be done is for the script to ask me for a name (string) -> Then convert it to uppercase -> then add it up character by character -> then have it do XOR 5678, and apply XOR 1234 to that result, and finally return the numerical value.

```
MisProgramasEnPython > KeyGenCrackmeCrushead1999.py > ...
1  usu = input('Indique nombre de usuario: ')
2  usu = usu.upper()
3  key = 0
4  if usu.isalpha() == True:
5      for c in usu:
6          key = key + ord(c)
7          key = key^int(0x444C)
8          print('Su serial es: ' + str(key))
9  else:
10     print ("El usuario debe contener solo letras")
11
12
13
14
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS COMENTARIOS

```
PS D:\Eugenio\Propio\Desarrollo\MisProgramas> & D:/Eugenio/Propio/AreaTrabajo/Desarrollo/MisProgramas/MisProgramasEnPython/key.py
Indique nombre de usuario: karasu
Su serial es: 17803
PS D:\Eugenio\Propio\Desarrollo\MisProgramas>
```

Correction (08/15/2024) Someone pointed out to me that if you use, for example, 'Zapallo' as the username (just to make a quick mention—with 'zapallo' the previous keygen works without any issues), when generating the serial with that initial keygen—it wouldn't work and would trigger the bad boy message.

Because of this, I started investigating why something had slipped past me, and indeed it had:

Upon tracing through it again, I realized that the problem is the Z.

	Hex dump	ASCII
E	5A 61 70 61 6C 6C 6F 00 00 00 00 00 00 00 00 00	Zapallo...
E	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

Notice that 'Z' is 5A in Hex -> well, because there is a line that complicates things:

00401385	. 84C0	TEST AL,AL
00401387	. 74 13	JE SHORT CRACKME.0040139C
00401389	. 3C 41	CMP AL,41
0040138B	. 72 1F	JB SHORT CRACKME.004013AC
0040138D	. 3C 5A	CMP AL,5A
0040138F	. 73 03	JNB SHORT CRACKME.00401394
00401391	. 46	INC ESI
00401392	. ^EB EF	JMP SHORT CRACKME.00401383
00401394	> E8 39000000	CALL CRACKME.004013D2
00401399	. 46	INC ESI
0040139A	. ^EB E7	JMP SHORT CRACKME.00401383
0040139C	> 5E	POP ESI
0040139D	. E8 20000000	CALL CRACKME.004013C2
004013A2	. 81F7 78560000	XOR EDI,5678
004013A8	. 8BC7	MOV EAX,EDI
004013AA	. ^EB 15	JMP SHORT CRACKME.004013C1
004013AC	> 5E	POP ESI
004013AD	. 6A 30	PUSH 30
004013AF	. 68 60214000	PUSH CRACKME.00402160
004013B4	. 68 69214000	PUSH CRACKME.00402169
004013B9	. FF75 08	PUSH DWORD PTR SS:[EBP+8]
004013BC	. E8 79000000	CALL <JMP.&USER32.MessageBoxA>
004013C1	> C3	RETN
004013C2	. \$ 33FF	XOR EDI,EDI
004013C4	. 33DB	XOR EBX,EBX
004013C6	> 8A1E	MOV BL,BYTE PTR DS:[ESI]

AL=5A ('Z')

There, when it compares and starts the loop to convert the characters to uppercase, etc.—'Z' (being 5A) being in AL—it is compared against 5A. This causes it, when it hits the instruction.

004013C4	. 33DB	XOR EBX,EBX	Registers (FP EAX 0000003A ECX 0012FDE4 EDX 7C91E4F4 EBX 00000000 ESP 0012FE98 EBP 0012FEB4 ESI 0040218E EDI 0012FF1C EIP 004013D4 C 0 ES 0023 P 1 CS 001B A 0 SS 0023 Z 0 DS 0023 S 0 FS 003B T 0 GS 0000 D 0 O 0 LastErr EFL 00010206 ST0 empty -?? ST1 empty -?? ST2 empty -?? ST3 empty -?? ST4 empty -NA ST5 empty 1.0 ST6 empty 1.0 ST7 empty 1.0
004013C6	> 8A1E	MOV BL, BYTE PTR DS: [ESI]	
004013C8	. 84DB	TEST BL, BL	
004013CA	. 74 05	JE SHORT CRACKME.004013D1	
004013CC	. 03FB	ADD EDI, EBX	
004013CE	. 46	INC ESI	
004013CF	. ^EB F5	JMP SHORT CRACKME.004013C6	
004013D1	> C3	RETN	
004013D2	\$ 2C 20	SUB AL, 20	
004013D4	. 8806	MOV BYTE PTR DS: [ESI], AL	
004013D6	. C3	RETN	
004013D7	. C3	RETN	
004013D8	\$ 33C0	XOR EAX, EAX	
004013DA	. 33FF	XOR EDI, EDI	
004013DC	. 33DB	XOR EBX, EBX	
004013DE	. 8B7424 04	MOV ESI, DWORD PTR SS: [ESP+4]	
004013E2	> B0 0A	MOV AL, 0A	
004013E4	. 8A1E	MOV BL, BYTE PTR DS: [ESI]	
004013E6	. 84DB	TEST BL, BL	
004013E8	. 74 0B	JE SHORT CRACKME.004013F5	
004013EA	. 80EB 30	SUB BL, 30	
004013ED	. 0FAFF8	IMUL EDI, EAX	
004013F0	. 03FB	ADD EDI, EBX	
004013F2	. 46	INC ESI	
004013F3	. ^EB ED	JMP SHORT CRACKME.004013E2	
004013F5	> 81F7 34120000	XOR EDI, 1234	
004013FB	. 8BDF	MOV EBX, EDI	
AL=3A (':')			
DS: [0040218E]=5A ('Z')			

Upon reaching SUB AL, 20 -> it causes 5A - 20 = 3A, and that means this is represented... the output would look like this:

Address	Hex dump	ASCII
0218E	3A 41 50 41 4C 4C 4F 00 00 00 00 00 00 00 00 00	: APALLO.....
0219E	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

So the script needs to be modified because it doesn't take that into account; therefore, it is modified so that when a name starting with 'Z' is used -> it replaces it with ':' and performs the operations from there.

Using 'Zorro' as the name, I made the changes to the script and got these results:

```
MisProgramasEnPython > KeygenCrackmeCRUSHEAD1999.py > ...
1 | import re
2 | usu = input('Indique nombre de usuario: ')
3 | permitido = r"[a-zA-Z0-9:]"
4 | usu = usu.upper()
5 | if re.match(f"^{permitido}+$", usu):
6 |     if usu[0] == 'Z':
7 |         usu = ':' + usu[1:]
8 |     key = 0
9 |     if usu:
10 |         for c in usu:
11 |             key = key + ord(c)
12 |             key = key^int(0x444C)
13 |             print('Su serial es: ' + str(key))
14 | else:
15 |     print ("El usuario debe contener solo letras")
16 |
17 |
18 |
19 |
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS COMENTARIOS

```
PS D:\Programas\Programacion\MisProgramas> & D:/Programas/Python/Python3
USHEAD1999.py
Indique nombre de usuario: Zorro
Su serial es: 17712
```

